PART 11 OF 18

Multi-Agent Security Architecture

A2A Communication Security, MCP Governance, Trust Negotiation, Federated Agents, Hierarchical Orchestration, Cross-Enterprise Collaboration

ENTERPRISE AI CONTROL ARCHITECTURE

Implementation Guide for Production AI Systems • 2026

11.1 Multi-Agent Architecture Security Fundamentals

Multi-agent systems introduce security challenges beyond those of single-agent deployments. When agents communicate, coordinate, and delegate to each other, the attack surface expands combinatorially: each communication channel is a potential injection vector, each delegation creates a potential trust boundary violation, and each shared resource is a potential lateral movement pathway. Multi-agent security requires explicit trust architecture, not implicit trust through shared infrastructure.

> ***Security Axiom for Multi-Agent Systems: An agent should never automatically trust another agent merely because they share the same infrastructure or were deployed by the same team. Every inter-agent interaction requires explicit trust verification as if the communication originated from an untrusted external source.***

11.2 Agent-to-Agent (A2A) Communication Security

11.2.1 Google A2A Protocol Security Analysis

Google's Agent2Agent (A2A) protocol (2025) provides a standardized mechanism for agent interoperability across vendors and platforms. A2A uses HTTP-based communication with structured task cards and agent discovery via Agent Cards published at well-known URLs. Security considerations for enterprise A2A deployments:

A2A Security Concern	Risk	Enterprise Control
Agent discovery (Agent Cards)	Malicious agent impersonation via spoofed Agent Card	Internal registry only; external A2A cards require vetting
Task delegation	Trust escalation through A2A task requests	Receiving agent inherits no more trust than sender; verify delegation scope
Push notifications (webhooks)	SSRF via malicious webhook URLs; webhook hijacking	Whitelist webhook targets; verify webhook signatures
Long-running tasks	State accumulation across extended A2A sessions	Session TTL limits; checkpoint-based state review
Streaming results (SSE)	Injection via malicious streaming content	Stream content scanning; structured output only
Authentication in base spec	A2A base spec uses HTTP+OAuth; implementation quality varies	Mandate mTLS for all enterprise A2A; add request signing

11.3 Trust Models for Multi-Agent Systems

11.3.1 Trust Level Assignment

When an agent receives a request from another agent, it must assign a trust level to that request. Trust level determines what actions the receiving agent will take on the request. Trust level should be based on verifiable cryptographic identity, not asserted claims.

Operator Trust

Internal agents deployed by the same operator team; mTLS certificate from internal CA + registered in agent registry. May give instructions within operator-defined limits.

User Trust

Agents acting on behalf of verified users but not part of operator deployment. Authenticated but limited to user-level permissions. Cannot override operator constraints.

None / Untrusted

Any agent not matching the above criteria—including agents claiming to be trusted but lacking cryptographic verification. Treated as untrusted external input.

11.3.2 Trust Verification Protocol

- **Mutual TLS:** All A2A connections require mTLS; both parties present X.509 certificates signed by trusted CA
- **SPIFFE SVID Verification:** Sending agent presents SPIFFE SVID in request header; receiving agent verifies against trusted trust domain
- **Request Signing:** All A2A requests signed with sending agent's private key; receiving agent verifies signature before processing
- **Delegation Token Verification:** If request includes a delegation claim, the delegation token is verified against the token's signing key and claimed delegator's identity
- **Trust Propagation Limits:** Receiving agent never grants more trust than the sending agent has; trust decrements at each hop

11.4 Hierarchical Agent Orchestration Security

11.4.1 Orchestrator-Agent Trust Architecture

In hierarchical multi-agent systems, an orchestrator agent coordinates multiple sub-agents. The orchestrator has broader context but must not have broader permissions than necessary. Sub-agents must maintain their own security posture independently of the orchestrator—a compromised orchestrator should not be able to instruct sub-agents to violate their own policies.

Component	Trust Level	Can Override Sub-Agent Policy?	Authorization
Human Principal	Ultimate Authority	Yes (within law/ethics)	Direct instruction; MFA verified

Component	Trust Level	Can Override Sub-Agent Policy?	Authorization
Orchestrator Agent	Operator Trust (if deployed by operator)	No	Can request; sub-agent enforces own policy
Sub-Agent	Scoped to own task	N/A	Enforces own policies regardless of orchestrator instruction
Tool	Tool trust score determines level	No	Executes tool function only; no policy authority

11.5 Cross-Enterprise and Federated Agent Security

11.5.1 Cross-Enterprise Agent Trust

As enterprises begin sharing AI capabilities and collaborating through multi-agent workflows that cross organizational boundaries, cross-enterprise trust becomes a critical architectural concern. Enterprise A agents may delegate tasks to Enterprise B agents, creating a cross-organizational permission chain that neither organization's standard IAM can fully govern.

11.5.2 Federated Trust Architecture

- **Cross-Domain SPIFFE:** Establish federated trust domains between enterprises; agents from Enterprise B are recognized in Enterprise A's trust domain with explicit trust limits
- **Cross-Enterprise Delegation Tokens:** Delegation tokens designed for cross-enterprise use; carry both source enterprise identity and receiving enterprise's trust scope
- **Data Sovereignty Controls:** Cross-enterprise agents must respect data sovereignty requirements; no PII or confidential data crosses organizational boundaries without explicit consent
- **Audit Trail Exchange:** Cross-enterprise transactions must generate mutually verifiable audit records that both organizations can independently validate
- **Incident Response Coordination:** Pre-negotiate incident response procedures for cross-enterprise agent security incidents; establish communication channels before they are needed

11.6 Shared Memory Security in Multi-Agent Systems

Shared memory creates a particularly dangerous cross-contamination risk in multi-agent systems. Information written by one agent—potentially from a compromised source—can influence the behaviour of other agents that read the same memory.

Risk	Attack	Control
Cross-agent injection	Agent A writes injection payload to shared memory; Agent B reads and executes	Memory content scanning; agent-specific read namespaces

Risk	Attack	Control
Information leakage	Agent A's confidential context leaks to Agent B through shared memory	Data classification enforcement; access control by data scope
Race condition exploitation	Attacker times writes to shared memory to influence agent decisions	Optimistic locking; atomic read-modify-write operations
Stale memory poisoning	Poison memory entry remains after source agent session ends	Memory provenance tracking; TTL on session-scoped entries